

High-Cost Claim Classifier

1. Prediction Question

This project predicts whether a beneficiary is likely to fall into the high-cost claims group, defined as annual medical expenditure greater than or equal to the training-sample 90th percentile. The app displays the model output as a live 0-100 risk score, risk tier, calibrated probability, intervention flag, and top drivers.

2. Data Source

The project uses CMS DE-SynPUF synthetic Medicare claims data, including beneficiary summary, inpatient, outpatient, carrier, and prescription drug event files. Beneficiary-level demographic, enrollment, utilization, chronic-condition, provider, and cost variables are transformed into a supervised beneficiary-year modeling dataset.

3. Architecture

The system uses a medallion pipeline with bronze, silver, and gold layers. Databricks and Spark perform distributed ETL and feature engineering. Delta tables store cleaned and model-ready datasets. MLflow tracks model training and artifacts. A FastAPI backend serves live predictions to a Streamlit web application.

4. Model Approach

The high-cost label is computed from annual claim cost, and the top-decile threshold is computed only from the training split before being reused for validation and test evaluation to avoid target leakage. The project trains and compares logistic regression, random forest, gradient boosting, and XGBoost models. Performance is evaluated using ROC-AUC, PR-AUC, recall, precision, F1 score, Brier score, calibration gap, and top-k capture/lift. Gradient boosting is used as the primary operational model with ROC-AUC = 0.8333, PR-AUC = 0.4653, and top-10 capture = 43.01%.

5. Learnings

The strongest lesson is that ranking performance and probability calibration are different problems. The models can identify high-risk beneficiaries reasonably well, but raw predicted probabilities must be interpreted carefully before being treated as actuarial risk estimates. The hardest engineering work was maintaining a clean end-to-end contract so Databricks training, MLflow artifacts, FastAPI serving, the Streamlit UI, and the grading test script all agreed on the same feature and response schema. If I rebuilt the project, I would put more time into formal calibration, automated retraining, and monitoring prediction drift after deployment.